Documenting Behavior

Well-written specs document how your system is intended to behave. Unlike a static documentation file, your specs are *executable*. You'll find out when they've grown stale, because you'll start seeing failures in the output. That means they're much easier to keep up to date than other forms of documentation.

RSpec encourages you to write examples that make great documentation. Its API favors plain-spoken descriptions of behavior, as in `An ideal sandwich is delicious`. Its rich expectation library helps you make it clear what the code you're testing is supposed to do. Its output formatters can even organize your specs' output into a coherent HTML document.

Transforming Your Workflow

Consider a world without test frameworks or BDD. Each feature you wrote would be a gamble. You'd fire up the app and poke around in the hopes of uncovering any obvious problems.

Driving your design from your specs completely transforms your workflow. Now, each run of your suite is an *experiment* you've designed in order to validate (or refute) a hypothesis about how the code behaves. You get fast, frequent feedback when something doesn't work, and you can change course immediately.

It's Fun!

Finally, driving your design from your specs is fun! Tackling a big problem all at once is hard and tends to give us a bad case of "programmer's block." TDD encourages us to break things down into small, achievable steps.

It's almost like a game: First, we come up with an example exposing a behavior the code doesn't yet implement. Then, we implement the behavior to make the specs pass. It's a constant stream of satisfaction.

Comparing Costs and Benefits

We hope you're convinced that BDD and RSpec can help you build good designs, and build them quickly. It's important to acknowledge the costs of what we do, whether we're talking about the minutiae of a single example or an entire approach to software development.

Writing Specs

Every spec takes time to write. That's why so many of the habits you've been practicing—outlining several examples at once, or creating reusable helper code, for instance—revolve around saving time.